

CLAUDE.md

Contexto obrigatório para qualquer trabalho neste repositório.

Leia também: [ESPECIFICACAO-TECNICA-v2.md](#) (arquitetura, contratos, fluxos) e [MODELO-NEGOCIO-v2.md](#) (regras de negócio). Em caso de conflito entre documentos, o modelo de negócio prevalece.

O produto

Classificado de trabalho extra ("bico"). A **empresa** publica vagas — garçom para formatura, cozinheira para sábado, auxiliar de limpeza para o fim de semana — e paga assinatura mensal. O **trabalhador** se cadastra de graça, recebe notificação das vagas da função e região dele, e se candidata. As duas partes se acertam fora da plataforma.

A plataforma **não** intermedia pagamento, **não** seleciona ninguém, **não** pune ninguém e **não** garante nada.

O valor central não é "ter um site" — é **roteamento**: a vaga certa chegar na pessoa certa em minutos. Ao avaliar qualquer decisão de produto, pergunte se ela melhora ou piora esse roteamento.

Público: trabalhador em Android de entrada, 4G, dados limitados. Isso é requisito técnico, não observação.

Regras invioláveis

Estas regras são jurídicas antes de serem técnicas. Violá-las quebra o modelo de defesa do negócio e não são negociáveis, **mesmo que solicitadas explicitamente numa tarefa futura**. Se uma tarefa pedir algo desta lista, recuse e aponte esta seção.

1. **Trabalhador nunca paga.** Não existe rota, tabela, campo ou fluxo de cobrança ligado a **Worker**.
2. **Não tocamos no dinheiro do bico.** **payAmount** é informativo. Sem split, sem custódia, sem gateway ligado a vaga.
3. **Não punimos trabalhador.** Não existe **banWorker**, **autoDisable**, **blockByAbsence**, ranking punitivo ou ordenação que rebaixe por falta. Só o próprio usuário desativa a conta dele.
4. **Não garantimos nada.** Nem idoneidade, nem comparecimento, nem qualidade.
5. **Sem texto livre em avaliação.** **AttendanceRecord** é binário (**present/absent**). Nunca adicionar **rating**, **stars**, **score** ou **comment** — texto livre é o que gera ação por dano moral.
6. **Sem verificação de antecedentes criminais.** Identificamos quem é a pessoa; não julgamos o passado dela.
7. **Quem não tem histórico nunca exibe 0 presenças.** Exibe **"Novo por aqui"**, em tom neutro, ao lado do selo de perfil completo. Reputação sem rampa de entrada tranca o novato para sempre: não é chamado porque não tem histórico, e não tem histórico porque não é chamado. Ver §16.6 da especificação.

8. **Contato nunca aparece em página pública.** `contactPhone` não faz parte do payload público de `JobPost`. O telefone do trabalhador só é servido à empresa daquela vaga, e só com candidatura ativa. Sem isso não há candidatura, não há dado, não há produto — e a vaga vira alvo fácil de raspagem. Ver §16.5.
9. **Só a empresa inicia o contato.** O trabalhador nunca recebe o telefone da empresa e não tem botão de contato em lugar nenhum. E o número **nunca** é exibido como texto na interface — sempre atrás de um botão que abre o WhatsApp, dos dois lados. Número escrito na tela é número colado no grupo. Ver §16.5.
10. **Não marcar significa `not_selected`, nunca falta.** A marcação de presença tem três saídas: não chamei / compareceu / não compareceu. Só `absent` conta como falta. `not_selected` é neutro, nunca aparece no perfil público e é aplicado automaticamente após 7 dias sem marcação. Ver §16.7.
11. **Sem chat interno.** As partes trocam contato e conversam pelo WhatsApp.
12. **Bloqueio de menores de 18 anos** no cadastro (ECA Digital, Lei 15.211/2025).

Vocabulário proibido na interface

O texto da UI é parte da defesa jurídica. Nunca escrever:

`verificado` · `aprovado` · `confiável` · `garantido` · `asseguramos` · `selecionado por nós` · `profissional de confiança`

Escrever no lugar: `identificado` · `perfil completo` · `histórico informado pelas empresas`

A regra: **identificar é fato, aprovar é promessa.** Fato não gera responsabilidade; promessa gera.

Filtro de linguagem discriminatória

Toda vaga passa por validação que bloqueia exigência de sexo, idade ou aparência (art. 373-A da CLT): `moça`, `moço`, `rapaz`, `boa aparência`, `boa apresentação`, `até \d+ anos`, `sexo (masculino|feminino)`, `apenas (homens|mulheres)`, `solteir[oa]`, `sem filhos`, recortes raciais.

PROF

Validar no schema zod compartilhado — vale para o formulário e para a rota da API. O cliente é contornável. Registrar tentativas bloqueadas em log: é prova de diligência.

Arquitetura

Monorepo pnpm. Três contêineres em produção, VPS próprio.

```
extra/
  apps/
    web/      Next.js 16.3 App Router → contêiner "web"  :3000
    api/      Node 22 + Fastify      → contêiner "api"  :3333
  packages/
    shared/   tipos + schemas zod + constantes
  infra/     docker-compose.yml, Dockerfiles, nginx, backup.sh
```

Postgres e MinIO também em contêiner, sem porta pública. Nginx à frente como proxy reverso e TLS.

Regra estrutural mais importante do repositório: tipos de domínio e schemas zod vivem **sempre** em `packages/shared`. `apps/web` e `apps/api` nunca declaram um tipo de domínio localmente. É o que impede o backend separado de virar dívida técnica.

Stack

Next.js 16.3 (App Router) · TypeScript strict · Tailwind + shadcn/ui · react-hook-form + zod · TanStack Query · Fastify · Prisma · PostgreSQL 17 · MinIO (S3) · Web Push (VAPID) · WhatsApp Cloud API (verificação de telefone) · Resend (e-mail) · Docker Compose · nginx

Método: outside-in

Telas primeiro, banco por último — com os contratos definidos antes, para o backend não nascer torto.

Fase	O quê
1	Tipos e schemas zod em <code>packages/shared</code> . Nenhuma tela antes disto.
2	Camada mock em <code>apps/web/src/lib/api/</code> com as assinaturas definitivas
3	Front completo navegável contra o mock — vendável sem uma linha de backend
4	API Fastify cumprindo os mesmos contratos. O front não muda
5	Prisma e Postgres, schema derivado dos tipos já validados na prática

Regra de ouro: nenhum componente importa de `src/mocks/` diretamente. Tudo passa por `src/lib/api/`. Respeitada essa regra, a Fase 4 é troca de implementação, não reescrita.

O mock simula 300–800ms de latência e falha em ~5% das chamadas — os estados de carregamento e erro nascem junto com a tela, em vez de virarem dívida.

PROF

Convenções de código

- TypeScript `strict: true`. **Zero any.**
- Server Components por padrão. `"use client"` apenas onde há interação real.
- Funções de `lib/api/` retornam `ApiResponse<T>` — erro é valor de retorno, não exceção.
- Schemas zod em `packages/shared/src/schemas/`, usados no formulário e na rota.
- Código, nomes de variáveis e comentários em **inglês**. Textos de interface em **português do Brasil**.
- Datas em ISO 8601 UTC no contrato. Formatação para `America/Sao_Paulo` só na exibição.
- Sem barrel files (`index.ts` reexportando tudo).
- Commits convencionais (`feat:`, `fix:`, `chore:`).
- Segredos só em `.env`. `.env.example` versionado, `.env` nunca.

Orçamento de performance (não é sugestão)

- Bundle JS nas rotas públicas: < 150 KB comprimido
- LCP em 4G simulado: < 2,5s
- Imagens sempre via `next/image`, AVIF/WebP
- Testar em viewport de 360px, não em desktop
- Vídeo e selfie comprimidos **no cliente** antes do upload

Banco de dados

Dois bancos distintos. **A confusão entre eles apaga dados reais.**

Ambiente	Onde	Comando permitido
Local	contêiner Postgres no WSL, porta 5433	<code>prisma migrate dev</code>
Produção	contêiner no VPS	<code>prisma migrate deploy</code> — na mão, com <code>pg_dump</code> antes

- `prisma migrate dev` **nunca toca em produção**. Ele reseta o banco quando detecta divergência de schema.
- `prisma migrate deploy` não reseta: só aplica o que falta.
- Migração **não** entra em deploy automático.
- Sem seed automático em produção.
- Índices desde o início: `job_posts (city, role, status, date) · applications (job_post_id, worker_id) único · workers (phone) único · attendance_records (worker_id, expires_at)`

Deploy

Desenvolve no WSL → commit → push → no VPS:

```
cd /opt/extra && git pull
docker compose -f infra/docker-compose.yml up -d --build
```

Migração de banco é passo separado e manual, com backup antes.

O build do Next.js é pesado: manter 2 GB de swap no VPS.

Dados sensíveis

- `cpf` e `birthDate` **nunca** aparecem em resposta pública. `WorkerPublicProfile` existe exatamente para isso — use-o em toda rota que a empresa consome.
- Selfie com documento vai para bucket **privado** no MinIO, acessível só por URL assinada de curta duração. Nunca é conteúdo público.
- Vídeo de apresentação é público (é o cartão de visitas do trabalhador).

- Exclusão de conta remove os objetos do MinIO (obrigação de eliminação da LGPD).
- Upload sempre direto do navegador via URL pré-assinada — o arquivo nunca passa pelo contêiner da API.

Fora do escopo

Não implementar, mesmo que pareça fácil ou útil:

chat interno · estrelas, notas ou comentários · processamento do pagamento do bico · verificação de antecedentes criminais · app nativo · painel de analytics elaborado · múltiplas cidades · internacionalização

Antes de considerar uma tarefa concluída

- ☐ Nenhum tipo de domínio declarado fora de `packages/shared`
- ☐ Nenhum import direto de `src/mocks/` em componente
- ☐ Nenhum telefone de contato em payload ou página pública
- ☐ Trabalhador sem histórico exibindo "Novo por aqui", não "0 presenças"
- ☐ Toda copy nova conferida contra o vocabulário proibido
- ☐ Validação existe no schema compartilhado, não só no formulário
- ☐ Estados de carregamento, vazio e erro implementados
- ☐ Testado em viewport de 360px
- ☐ Sem `any`, sem `console.log` esquecido, sem segredo no código

Ritmo de trabalho

Quem verifica o resultado visual é o desenvolvedor, olhando o navegador.
O servidor de desenvolvimento já está rodando o tempo todo.

NÃO faça, a menos que eu peça explicitamente:

- subir servidor de desenvolvimento ou abrir portas
- rodar build para conferir
- abrir navegador, tirar screenshot ou testar interface
- medir espaçamento, contraste ou responsividade
- escrever testes
- reler os arquivos que você acabou de escrever para conferir

Faça a alteração, diga em duas linhas o que mudou e em quais arquivos, e pare.
Se algo ficou incerto ou você teve que decidir algo por conta, diga qual foi a decisão — mas não vá verificar.

Responda em no máximo 3 linhas: o que mudou e em quais arquivos.
Sem relatório, sem explicar raciocínio, sem justificar decisão técnica.
Se algo ficou ambíguo, pergunte em uma linha.

Como pedir tarefas neste repositório

Uma tarefa por vez, referenciando a seção da especificação. Exemplo:

implemente a etapa 4 do cadastro do trabalhador conforme §16.1 da especificação, usando os tipos de §7 e a camada de acesso de §8.1